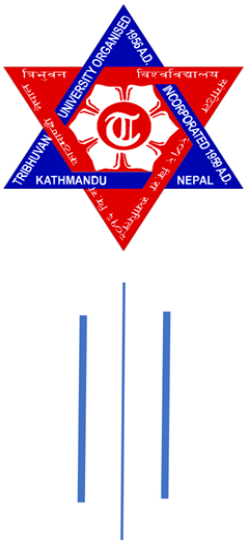
TRIBHUVAN UNIVERSITY

INSTITUTE OF SCIENCE AND TECHNOLOGY

AMRIT SCIENCE CAMPUS

Lainchaur, Kathmandu



Software Engineering Project Report

B.Sc. CSIT/Third Year/Sixth Semester

Project Title: Hotel Management System (HMS)

Submitted By:

Arun Chaudary

Bibek Kafle

Jenish Ghimire

Dipak Tamang

Faculty: CSIT

Section: A

Submitted To: **Mr.**Chakra Narayan Rawal Sir

Internal Examiner

Signature: _____

External Examiner

Signature: _____

Abstract

The Hotel Management System (HMS) is a software solution developed to automate and streamline hotel operations, improving efficiency and guest satisfaction. It manages core functions such as room reservations, check-in and check-out, billing, and inventory control through a centralized platform, reducing manual errors, administrative workload, and coordination issues. In the context of Nepal's growing tourism industry, where many small and mid-sized hotels still rely on manual processes, the proposed system addresses inefficiencies, delays, and customer dissatisfaction by supporting real-time data access, scalability, and integration with online booking systems. The HMS also enhances reporting accuracy, prevents overbooking, and optimizes operational performance, enabling hotels to improve staff productivity and remain competitive in a dynamic hospitality market.

Keywords: *Hotel Management System, Automation, Room Reservation, Tourism Industry in Nepal, Operational Efficiency, Guest Satisfaction*

TEAM COLLABORATION

Our team collaborated effectively to complete the Hotel Management System project. Jenish served as Team Leader and Backend Developer, managing tasks, backend development using Django, and overall progress.Arun worked as UI/UX Developer, designing the interface and completing Chapters 1 and 2. Deepak, the QA Engineer, handled testing (Chapters 3 and 6). Despite facing laptop issues while in his village, he later completed all testing tasks.Bibek worked as Frontend Developer, completing Chapters 5 and 7 and ensuring smooth integration between frontend and backend.

Communication and coordination were mainly done through Viber, while Google Docs was used for report writing and GitHub for code collaboration.Although some members faced delays, the team leader ensured steady progress through regular follow-ups and motivation.In the end, all members contributed to the successful completion of the project despite challenges.

TABLE OF CONTENT

List of Figures

Appendix : UI Screenshots

CHAPTER 1

INTRODUCTION

A Hotel Management System (HMS) is an integrated software platform designed to streamline and automate the daily operations of hotels. Its primary purpose is to efficiently manage a variety of tasks, including reservations, guest check-ins and check-outs, billing, and commercial housekeeping, all within a centralized system. By automating routine operations, an HMS minimizes human errors and reduces staff workload. Real-time data analytics enable hotel managers to monitor performance, make informed decisions, and optimize revenue. Cloud-based access ensures seamless management across multiple properties and allows for remote supervision. Overall, implementing an HMS enhances guest satisfaction and significantly improves operational efficiency.

1.1 Overview

Hotel Management Systems (HMS) are essential software platforms designed to efficiently manage hotel operations while enhancing guest experiences. In Nepal, a country with significant tourism potential, these systems play a crucial role in accommodating increasing tourist numbers and addressing operational challenges.

An HMS streamlines processes such as reservations, check-ins, billing, and housekeeping, reducing manual workload and minimizing errors. Real-time data analytics enable hotel managers to monitor performance, optimize revenue, and make informed decisions. Cloud-based solutions allow for seamless management of multiple properties and remote oversight, ensuring operational continuity.

The adoption of HMS not only improves operational efficiency but also enhances guest satisfaction by providing faster services, accurate information, and personalized experiences. According to the **National Hotel and Restaurant Survey 2080** published by the National Statistics Office (NSO), the hotel and restaurant sector contributed approximately **2% of Nepal's GDP** in the fiscal year 2022/23, highlighting its importance to the national economy and the need for technological modernization to support sustainable growth.

1.2 Problem Statement

Hotels in developed countries widely use advanced Hotel Management Systems (HMS) to integrate reservations, front desk operations, housekeeping, billing, and real-time data analytics, significantly improving operational efficiency and guest satisfaction. In contrast, many hotels in Nepal, particularly small and mid-sized establishments still rely on manual and outdated processes, leading to frequent errors, slow reservation handling, double bookings, extended check-in and check-out times, and inconsistent service quality. Additional challenges such as full upfront payment requirements, limited skilled labor, and minimal technological adoption further reduce staff efficiency and guest experience. These operational inefficiencies are compounded by external factors including economic fluctuations, seasonal tourism, and socio-political instability, which increase uncertainty and revenue volatility. Together, these issues highlight the urgent need for a centralized and automated Hotel Management System in Nepal to streamline operations, reduce manual errors, and enhance competitiveness in the growing tourism market.

1.3 Objectives

The primary objective of this study is to design an automated Hotel Management System (HMS) tailored for Nepalese hotels, addressing operational inefficiencies caused by manual processes and enhancing overall hotel performance.

The specific objectives are:

- To streamline hotel operations by automating key processes such as room reservations, check-in/check-out, billing, and inventory management, reducing manual errors and improving efficiency.
- To enhance guest experience by providing faster service, accurate real-time information, and a more organized workflow that minimizes delays and improves customer satisfaction.
- To improve staff productivity by offering real-time access to operational data, enabling staff to manage room availability, guest preferences, and resources more effectively.

1.4 Literature Review

Hotel Management Systems (HMS) are critical to improving operational efficiency, service quality, and managerial decision-making within the hospitality industry through the automation of core functions such as reservations, billing, inventory control, housekeeping coordination, and performance reporting [19]. Globally, advanced HMS solutions enable real-time data access, reduce manual errors, prevent overbooking, and support data-driven strategic planning, resulting in enhanced guest satisfaction and staff productivity [19]. In Nepal, the rapid expansion of the tourism sector, along with the presence of international hotel chains such as Marriott International in major urban centers, reflects a growing demand for modern hotel management practices [8]–[10]. Despite this progress, a significant number of small and mid-sized hotels continue to depend on manual processes or outdated software systems, leading to operational inefficiencies, delayed service delivery, limited transparency, and inconsistent guest experiences [9]–[12]. Furthermore, socio-political disruptions, including the Gen Z protests, have exposed the vulnerability of Nepal's hospitality sector and highlighted the importance of resilient and adaptable digital systems [14]. Although international literature strongly validates the effectiveness of automated HMS solutions, limited research focuses on systems tailored to Nepal's unique operational, infrastructural, and socio-political context. This gap emphasizes the need for locally customized, automated HMS solutions capable of improving operational transparency, ensuring service consistency, and supporting sustainable growth within Nepal's hotel industry [13].

1.5 Report Organization

The report is structured to provide a comprehensive understanding of the design and implementation of an automated hotel management system tailored for Nepalese hotels. The following sections outline the organization of the report, ensuring all aspects of the case study are covered comprehensively, with a focus on local challenges and opportunities.The executive summary serves as a concise overview, highlighting the main goals and results of the case study. It briefly outlines the objective of designing an automated hotel management system to address operational inefficiencies in Nepalese hotels, summarizing key findings and recommendations for adoption. This section is crucial for busy stakeholders, providing a quick snapshot of the report's significance.

CHAPTER 2

BACKGROUND AND CONTEXT

2.1 Industry Background

Nepal's hotel industry is thriving, driven by its stunning natural landscapes including the Himalayas and its rich cultural heritage, such as UNESCO World Heritage sites. According to the **Nepal Tourism Board**, foreign tourist arrivals reached **1,147,567 in 2024**, marking a **13.1% increase** from 1,014,882 in 2023. This robust post-COVID-19 recovery underscores Nepal's appeal as a destination for both adventure and cultural tourism. With plans to attract **1.5 million visitors in 2025**, the hotel sector faces increasing pressure to scale operations efficiently to meet growing demand.

Historically, the hotel sector in Nepal experienced a compound annual growth rate **(CAGR) of 7–10%** prior to the pandemic. Future projections indicate an expected growth rate of approximately **8.09% annually between 2024 and 2029**, fueled by improved infrastructure, digital adoption, and industry reforms aimed at enhancing service quality and competitiveness.

However, industry growth is heavily influenced by socio-political and economic factors. Political instability, such as strikes, policy changes, or government transitions, can disrupt hotel operations and reduce tourist inflows. Social movements and youth-led protests, particularly in urban centers like Kathmandu and Pokhara, may temporarily lower occupancy rates and cause cancellations, affecting operational efficiency and revenue. Additionally, seasonal tourism fluctuations during off-peak monsoon and winter periods along with global economic downturns, further impact hotel revenues and operational planning.

2.2 Current System

Many hotels in Nepal, particularly small and mid-sized establishments, still rely on manual processes or outdated software to manage daily operations. Common practices include:

- **Paper-Based Records**: Reservations, guest check-ins, and check-outs are often recorded manually, requiring staff to continuously update records by hand

- **Limited Technology Adoption**: Hotels heavily depend on phone calls, emails, or in-person communication rather than automated digital tools, limiting operational efficiency.
- **Manual Inventory Management**: Room availability, supplies, and amenities are tracked using spreadsheets or physical logbooks, making real-time updates difficult.
- **Basic or Legacy Software:** Some properties use software for billing and accounting, but these systems often lack integration with modern platforms such as Online Travel Agencies (OTAs) or Point-of-Sale (POS) systems, restricting streamlined operations.

For instance, Hotel Hana in Thamel, Kathmandu, previously managed all operations manually, including updating OTA listings and processing guest transactions through disconnected systems. This reliance on manual or outdated methods is widespread, particularly in areas where technology access is limited or budget constraints prevent software upgrades.

2.3 Limitations of the Current System

The reliance on manual processes and outdated software presents several significant challenges for hotels in Nepal:

- **Time-Consuming Operations**: Manual booking, billing, and inventory Management requires substantial effort, diverting staff from guest-facing tasks. For example, Hotel Heranya's team spent hours manually updating OTA listings, slowing overall operations and reducing service efficiency.
- **Frequent Overbooking:** Without real-time integration between booking platforms and internal systems, double bookings are common, leading to guest dissatisfaction and operational disruption.
- **Inefficient Reporting**: Paper-based systems or basic software cannot generate detailed, real-time reports, limiting management's ability to analyze trends, optimize pricing, or enhance service quality.
- **Scalability Issues:** With rising tourist arrivals, manual systems struggle to handle high volumes efficiently, restricting hotels' growth potential and operational Expansion.
- **Error-Prone Processes:** Manual data entry increases the likelihood of mistakes in

billing, inventory tracking, and guest records, negatively impacting guest satisfaction and hotel reputation.

- **Vulnerability to External Factors:** Hotels remain highly susceptible to socio-political and economic disruptions, including political unrest, strikes, youth-led protests, seasonal tourism fluctuations, and global economic downturns, which can further exacerbate operational challenges and revenue instability.

2.4 Need for Automation

The challenges posed by manual processes and outdated systems highlight the urgent need for automation in Nepalese hotels. Implementing a modern Hotel Management System (HMS) offers several key benefits:

- **Streamlined Operations**: Automated systems can manage reservations, check-ins/check-outs, billing, and inventory in real-time, reducing manual workloads and allowing staff to focus on guest services.
- **Prevention of Overbooking:** Integration with Online Travel Agencies (OTAs) and internal systems ensures accurate, real-time room availability, minimizing double bookings and improving guest satisfaction
- **Enhanced Reporting and Analytics:** Automated systems can generate detailed, real-time reports on occupancy, revenue, and guest preferences, enabling management to make data-driven decisions and optimize pricing and services
- **Scalability:** Cloud-based HMS solutions allow hotels to manage multiple properties, handle increasing tourist arrivals, and expand operations without major infrastructure changes.
- **Error Reduction:** Automation minimizes manual data entry errors in billing, inventory management, and guest records, improving accuracy and reliability.
- **Resilience to External Factors:** Automated systems provide flexibility and operational continuity in the face of political instability, strikes, social protests, seasonal fluctuations, and economic downturns, helping hotels maintain service quality and revenue stability.
- **Improved Guest Experience:** Fast, accurate services and personalized experiences enhance guest satisfaction, encourage repeat visits, and foster positive reviews.

CHAPTER 3

SYSTEM ANALYSIS

3.1 System Analysis

System Analysis is the process of studying an existing system to understand its problems, needs, and expectations so that an improved system can be designed. For the Hotel Management System (HMS), requirements collection was carried out to ensure that the system meets real hotel operational needs and user expectations.

To collect accurate and reliable requirements, the following methods were used:

1. Hotel On-Site Visit

An on-site visit was conducted to observe real hotel operations and understand how daily activities are performed. Hotels such as Aloft Hotel, Chinese restaurants, and local Nepali tourist hotels were visited.

Through direct observation, the following information was gathered:

- Understanding daily workflows such as room reservations, check-in/check-out, billing, and room allocation
- Identifying problems, inefficiencies, and delays in existing manual or semi-automated systems
- Observing how hotel staff interact with current tools or manual records
- Gaining knowledge of the physical environment where the HMS will be used

This method helped ensure that the system requirements are realistic, practical, and aligned with actual hotel operations.

2. Interviews and Questionnaires

Structured and semi-structured interviews were conducted with hotel personnel, including front-desk staff, managers, housekeeping supervisors, and IT personnel. Through these interviews, information was gathered about:

- Challenges with the current process or system
- Required features and functionalities of the new HMS
- Security concerns and data management needs
- User preferences, system expectations, and workflow limitations

These insights helped clarify functional and non-functional requirements from a user perspective.

3. Online Sources and Hotel Websites

Research was conducted using hotel websites and online resources to understand current industry standards and trends.

This analysis provided:

- Feature comparison with modern hotel management systems
- Understanding of online reservation processes, room management, and digital services
- Insights into UI/UX design trends in hospitality software
- Knowledge of competitors' strengths and weaknesses

This research helped refine system requirements to ensure that the HMS is competitive, user-friendly, and up to date with industry standards.

3.1.1 Requirements Analysis

System requirements refer to the specifications or criteria that a computer system, software, or application needs to function properly. These requirements detail the hardware, software, and network resources that are necessary to install, run, and support the system or application effectively. Usually supplied by the program provider, these specifications include both minimum and recommended values.

i. Functional Requirements

Functional requirements describe the specific behaviors, actions, and functionalities that a system, software, or product must be able to perform. They detail what the system should do in order to meet user needs, and they define the features and operations that the system must support to function correctly and deliver the desired outcomes. The following functional requirements define the specific behaviors and services the system must provide:

1. User

- **User Authentication (Login)** The system shall allow registered guests to log in by providing valid credentials (username and password) to access personalized features.
- **Browse Hotel Portfolio** The system shall provide a public interface for users to view comprehensive hotel information, including available room types, facilities, and services offered.
- **Real-time Room Booking** The system shall allow logged-in users to select specific room types and dates, check real-time availability, and confirm reservations.
- **Booking History Management** The system shall allow authenticated users to access a private dashboard to view a historical log of their past and active room bookings.
- **Inquiry and Feedback (Contact Form)** The system shall provide a contact form allowing both guests and unregistered visitors to submit inquiries, messages, or feedback directly to the hotel management.

2. Administrator

- **Administrative Authentication** The system shall provide a secure login gateway for administrators to access the backend management interface with elevated privileges.
- **Dashboard Overview** The system shall provide a centralized dashboard displaying operational summaries and quick access to all administrative control modules.
- **Booking Management** The system shall allow administrators to view the complete list of bookings, modify booking statuses, and perform cancellations or updates as required.
- **Communication Management** The system shall allow administrators to view incoming contact messages from users, respond to inquiries, and track the status of resolved messages.

- **Room and Inventory Management** The system shall allow administrators to update room details (pricing, descriptions) and manually override room availability status.
- **User Activity Monitoring** The system shall maintain activity logs, allowing administrators to monitor user actions and manage user accounts to ensure system security and integrity.

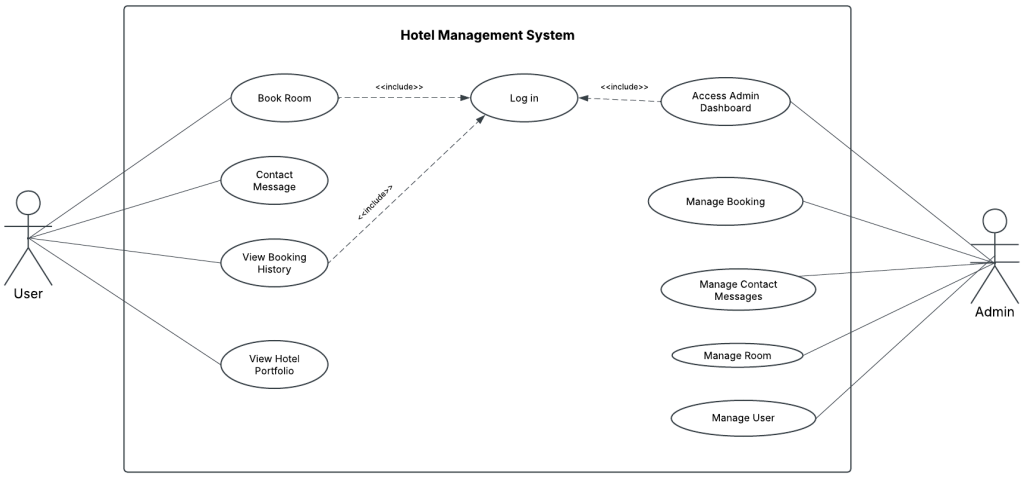


Figure 3.1 : Use Case Diagram

ii. Non-functional Requirements

Non-functional requirements (NFRs) describe how a system performs its tasks rather than what tasks it performs (which are covered by functional requirements). They define the quality attributes, constraints, and performance standards the system must meet to ensure it works effectively in the real world.

In other words, non-functional requirements set expectations about the system's behavior, focusing on its performance, security, usability, reliability, scalability, and other operational characteristics.

1. Backup & Security Requirements

A. Backup Requirements

- **Database Backups:**
    - Automatic backups available (paid Render plans)

    - Manual backups using pg_dump for free plan users
- Frequency: Daily or weekly backups stored on Google Drive / OneDrive
- Disaster Recovery: Backup files can restore a new Render DB instance at any time

B. Security Requirements

Data Encryption

- All communication encrypted via HTTPS (TLS 1.3)
- Passwords hashed using PBKDF2 + SHA256 (Django default)
- Sensitive data encrypted and stored securely

Role-Based Access Control (RBAC)

- **Admin:** Full management access
- **Staff:** Limited access to bookings and room management
- **Guest/User:** Can book rooms and manage their own profile
- Unauthorized access attempts are automatically blocked

Authentication & Multi-Factor Security

- Django Authentication Framework for login and session handling
- Optional MFA for admin users (django-otp, Google Authenticator)
- Secure + HttpOnly cookies to prevent session hijacking

Protection for Data in Transit & At Rest

- HTTPS for all data transmission
- PostgreSQL secure storage for data at rest
- Environment variables securely stored in Render (SECRET_KEY, DB URL)

CSRF & XSS Protection

- Django CSRF tokens enabled
- Input validation and HTML escaping to block XSS attacks

Backup Security

- Backups stored in secure locations (cloud or local)
- Accessible only to authorized administrators

Session & Logout Management

- Automatic session expiration after inactivity
- Immediate invalidation of session cookies on logout

2. Performance Requirements

- The system must handle up to 500 concurrent user sessions during peak times.
- Search queries should return results within 2 seconds or less.

3. Scalability Requirements

- The system must scale to support 100 users or more depending on hotel growth.
- The database must support increasing traffic, reservations, and user data.

4. Reliability & Availability Requirements

- System uptime must be 99.9%, with less than 1 hour of downtime per month.
- The system must recover from failures within 5 minutes.

5. Usability Requirements

- The system must be usable by non-technical staff with minimal training.
- Receptionists should complete guest check-in within 3 minutes.
- Interfaces must be intuitive, responsive, and mobile-friendly.

6. Maintainability Requirements

- Codebase must be modular, documented, and easy to update.
- Adding new features (e.g., new room types or services) must not require major downtime.

7. Compatibility Requirements

- Must integrate with existing hotel POS systems and third-party services.
- Compatible with Windows, macOS, Linux, Android, and iOS devices.

8. Portability Requirements

- The system should run on both on-premises servers and cloud platforms (AWS, Render, Azure) without major changes.

9. Compliance & Regulatory Requirements

- Must comply with PCI-DSS for payment security.
- Must follow GDPR and local data protection laws for guest information.

10. Backup & Recovery Requirements

- Daily automated or manual backups of critical system data.
- The system must restore data from backups within 30 minutes in case of data loss.

3.1.2 Feasibility Analysis

A feasibility study is an analysis of the viability of a proposed project or system. It assesses whether a proposed solution is technically, financially, and operationally viable, and whether it is likely to succeed. The goal of a feasibility study is to determine whether the project should move forward, and if so, what the best approach would be.

Types:

i. Technical Feasibility:

a. Technology Requirements:

- The system must be capable of handling reservations, billing and guest and management
- The system should be designed with modern web technologies, such as HTML5, CSS3, JavaScript, and backend technologies python (Django).
- The database would be a relational database PostgreSQL which can manage large datasets for reservations, guest preferences, billing, etc.
- The system must be accessible via web browsers for customer and admin page for centralized control for hotel manager, IT staff or concerned authority.

b. Technical Challenges:

- Security: Protecting sensitive data, such as guest information and payment details, through encryption and secure authentication methods.
- Scalability: As the hotel may expand, the system must be able to scale to support new rooms, services, and possibly multiple properties.

c. Expertise Needed:

- A team of developers skilled in web technologies.
- Database administrators for designing and maintaining the database.
- IT professionals, security chief to handle server infrastructure, cloud services, and security.

Justification: The system is technically feasible as it is implemented using Django and PostgreSQL with cloud deployment on Render. These technologies provide security, scalability, and reliable performance. The development was completed using existing technical skills and tools.

ii. Operational Feasibility:

a. Operational Efficiency:

- The system must streamline operations like room booking, guest check-in/check-out, billing, and admin dashboard. By automating many of these tasks, the system will allow hotel staff to focus on guest service, improving overall operational efficiency.
- It must be intuitive and easy to use for hotel staff, who may vary..

b. Staff Training:

- Hotel staff will need training to use the system effectively. Training must be quick and not disrupt daily operations.
- Staff should be able to manage reservations, process payments, update room availability, and handle guest requests seamlessly.

c. Adaptability:

- The system should be flexible enough to accommodate various hotel sizes and configurations, including different room types, rates, and seasonal adjustments.

Justification: The system automates hotel operations such as booking, billing, and check-in/check-out, making daily tasks faster and more accurate. Its user-friendly interface allows staff to use the system with minimal training.

iii. Economic Feasibility:

a. Costs:

- Development Costs: Hiring developers, purchasing licenses for software, and acquiring hardware infrastructure (servers, workstations, etc.).
- Ongoing Maintenance Costs: Regular updates, bug fixes, and system maintenance.
- Hosting Costs: There will be ongoing costs for cloud hosting as the model is developed on render for now it is free but for commercial products have to switch into paid subscriptions.
- Training and Support: Staff training for using the system and providing ongoing customer support.

b. Revenue Generation:

- Improved Efficiency: Automating tasks such as reservations, billing, and guest management reduces labor costs and increases operational efficiency.
- Increased Guest Satisfaction: A well-functioning system can enhance the guest experience, potentially leading to higher repeat business and positive reviews, which may increase revenue.
- Better Data-Driven Decisions: The system will provide analytics that can help improve pricing strategies, marketing efforts, and operational efficiencies, leading to increased profitability.

c. Cost-Benefit Analysis:

- Initial Costs: Development, hardware, and software expenses may be high initially.
- Long-Term Savings: Reduced manual labor, faster check-in/check-out times, and fewer booking errors will lead to cost savings in the long term.

Justification: The system is economically feasible because it uses open-source tools and low-cost cloud hosting. Automation reduces manual effort and operational costs, providing long-term financial benefits.

iv. Schedule Feasibility:

a. Development Timeline:

- A hotel management system typically takes 10-16 months to develop, depending on complexity and the number of features to be integrated.
- A phased implementation approach may be considered, starting with core functionality like reservations and billing, followed by additional features like guest services and reporting.
- But, the MVP of this product was made within 1 month of hardcore development with all ui,frontend,backend and testing done parallely with agile methodology.

b. Testing and Deployment:

- Adequate time should be allocated for testing the system's functionality, security, and performance.
- Deployment will require coordination with hotel staff for proper training and transition to the new system.

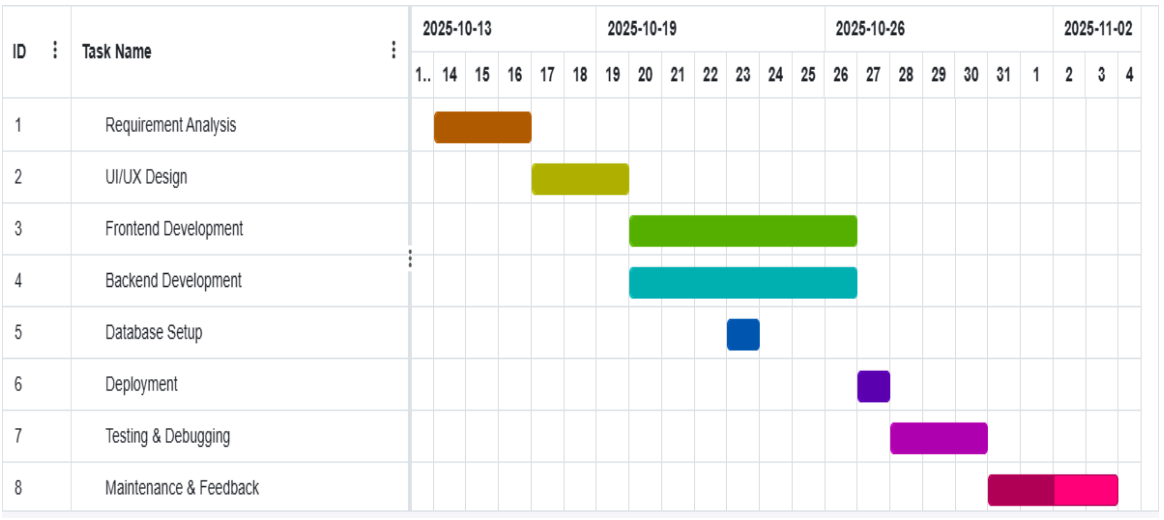


Figure 3.2 : Gantt Chart

Justification: The system is schedule feasible as a working MVP was developed within one month using an agile approach. Parallel development and testing ensured timely completion without compromising quality.

CHAPTER 4

SYSTEM DESIGN

4.1 High Level Architecture

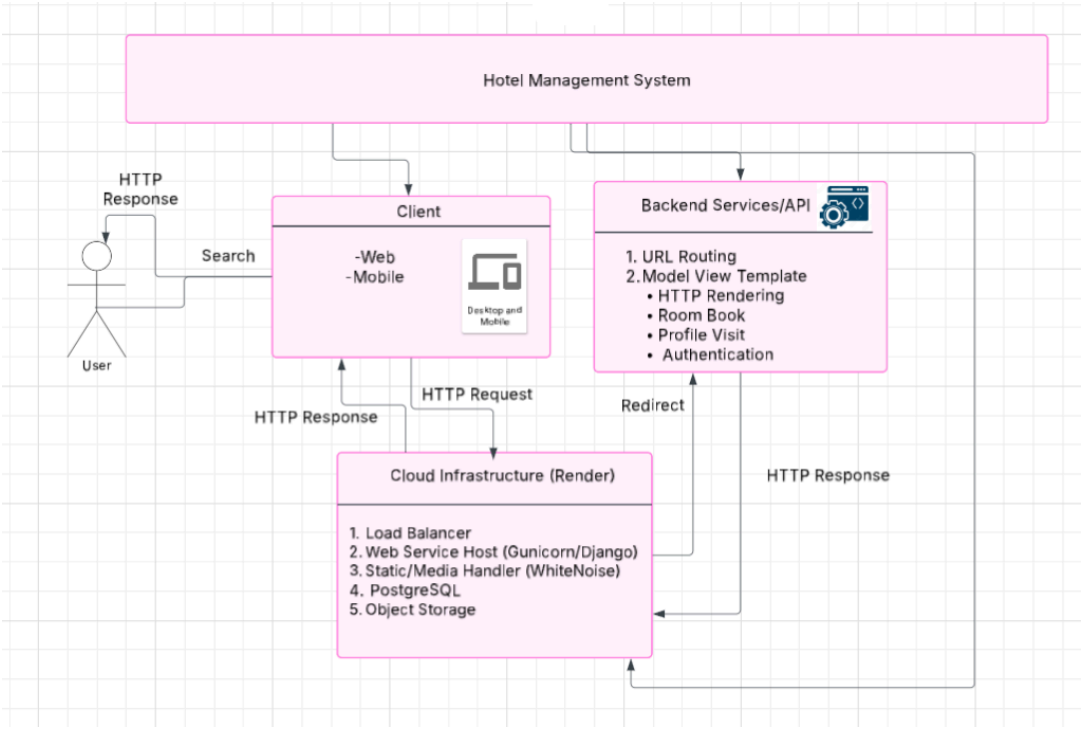


Figure 4.1: High Level Architecture

4.2 Class Diagram

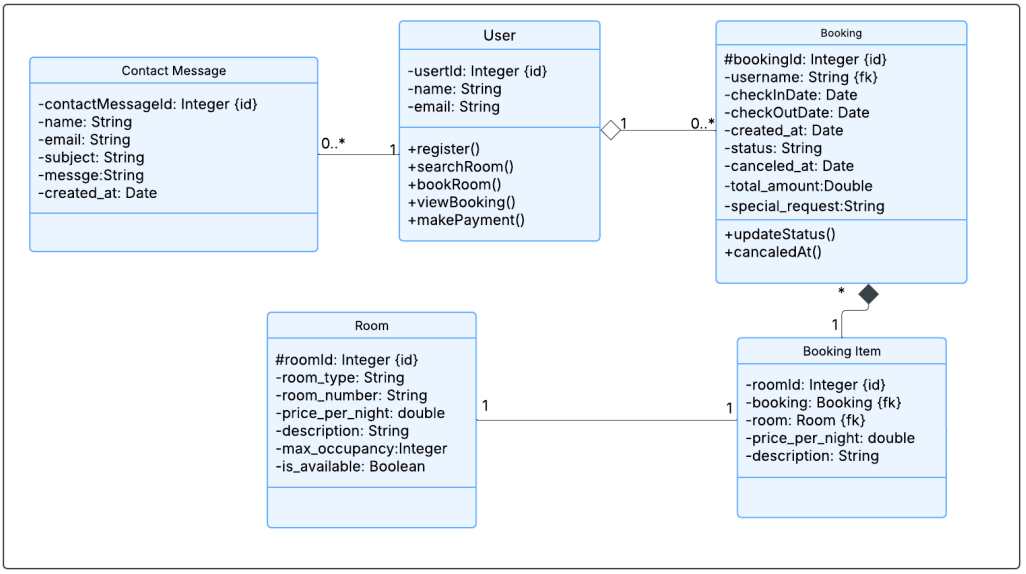


Figure 4.2: Class Diagram

4.3) Activity Diagram

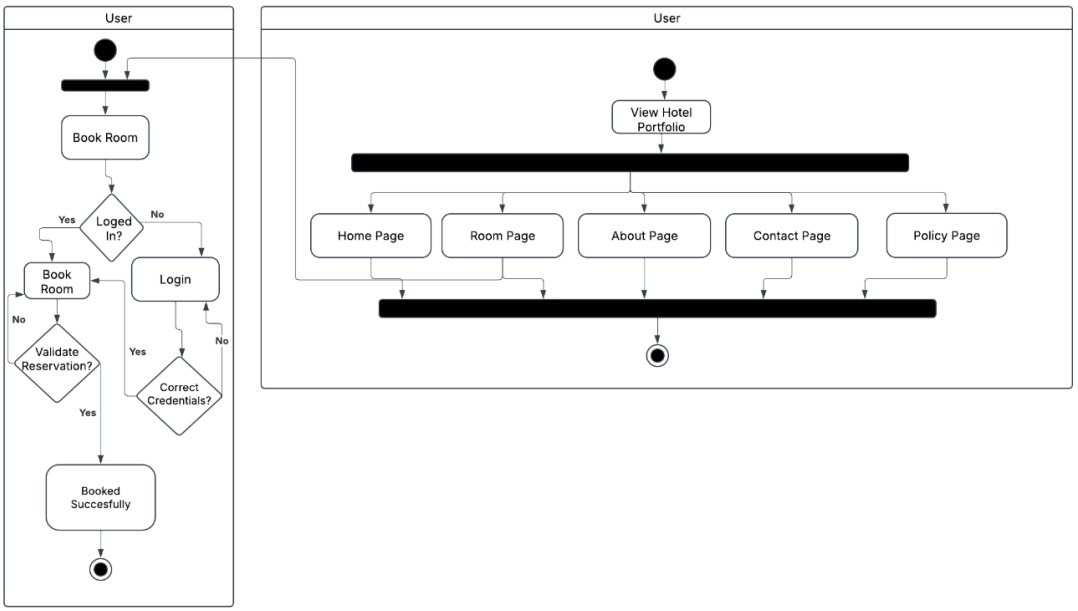


Figure 4.3: Activity Diagram

4.4 Sequence Diagram

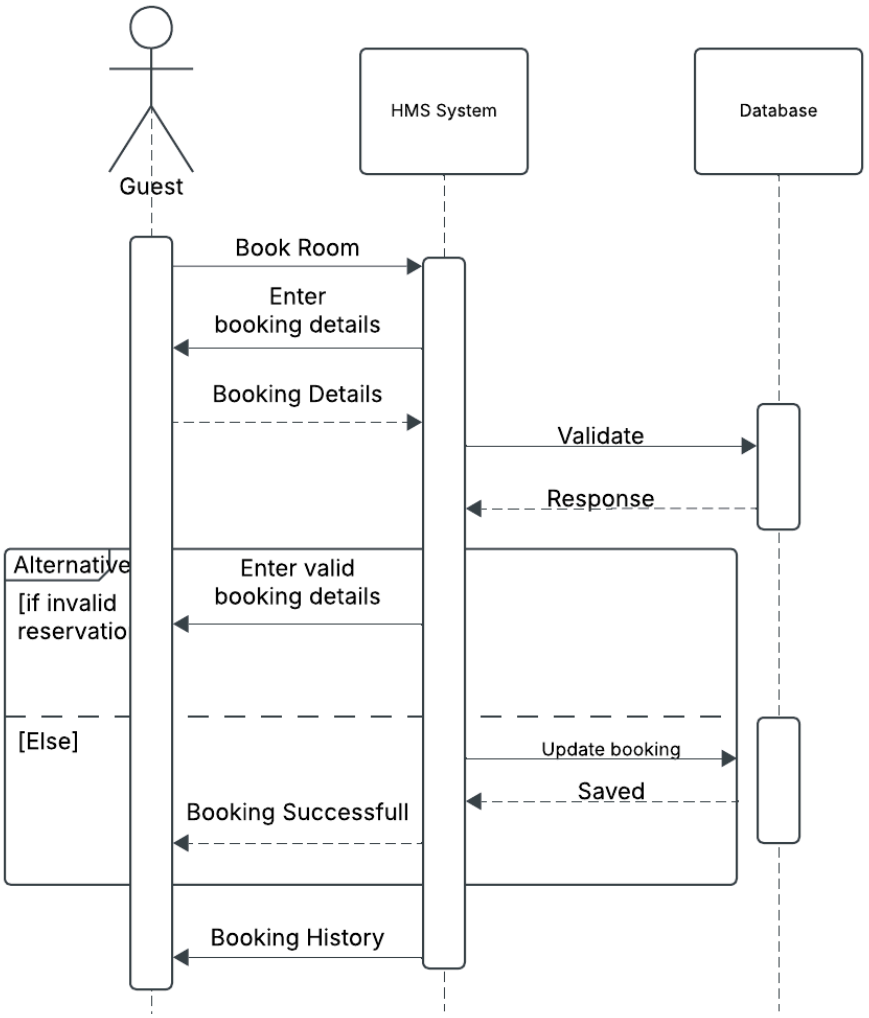


Figure 4.4: Sequence Diagram(Room Booking)

4.5 Wireframe

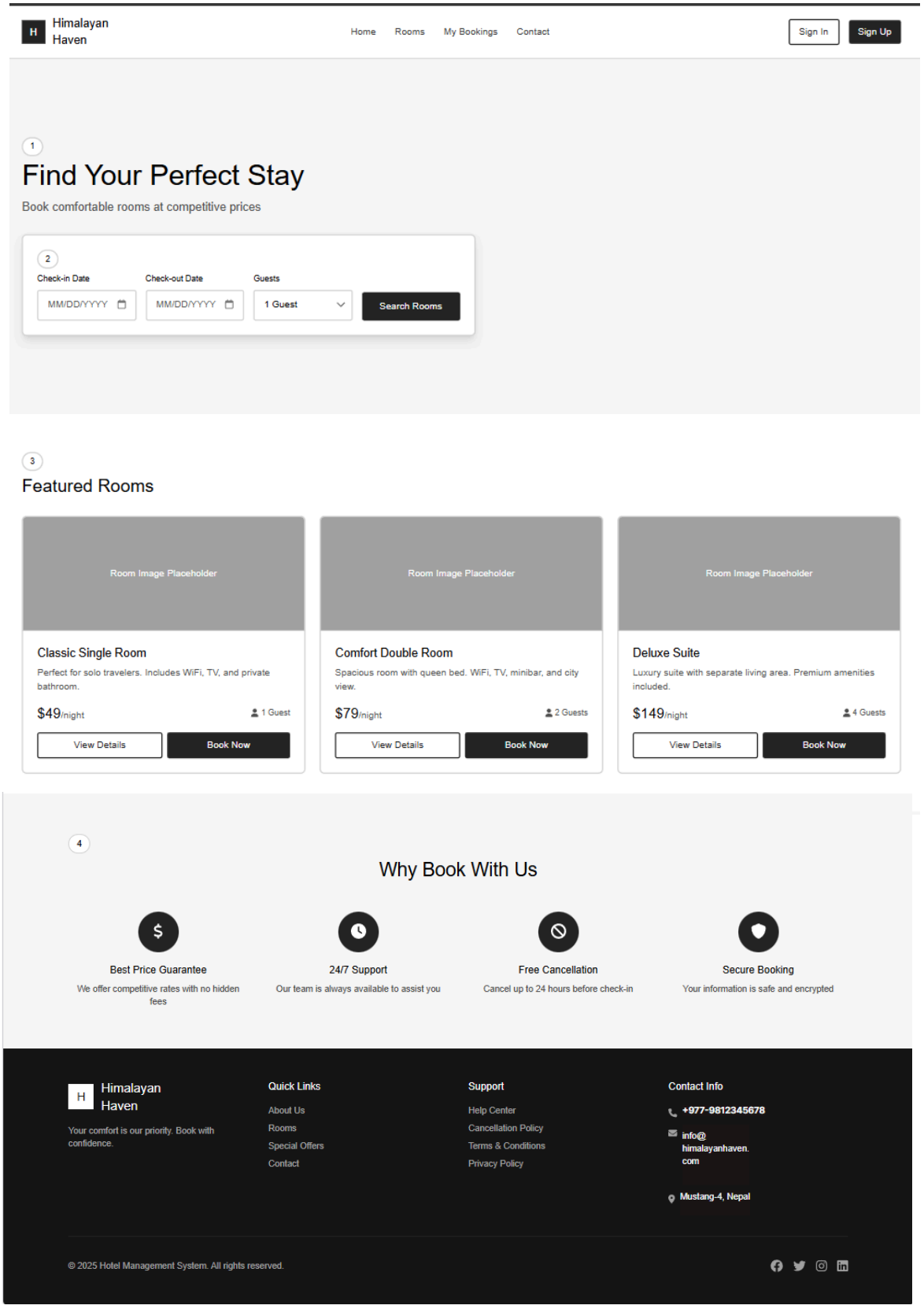


Figure 4.5.1: Home Page Wireframe

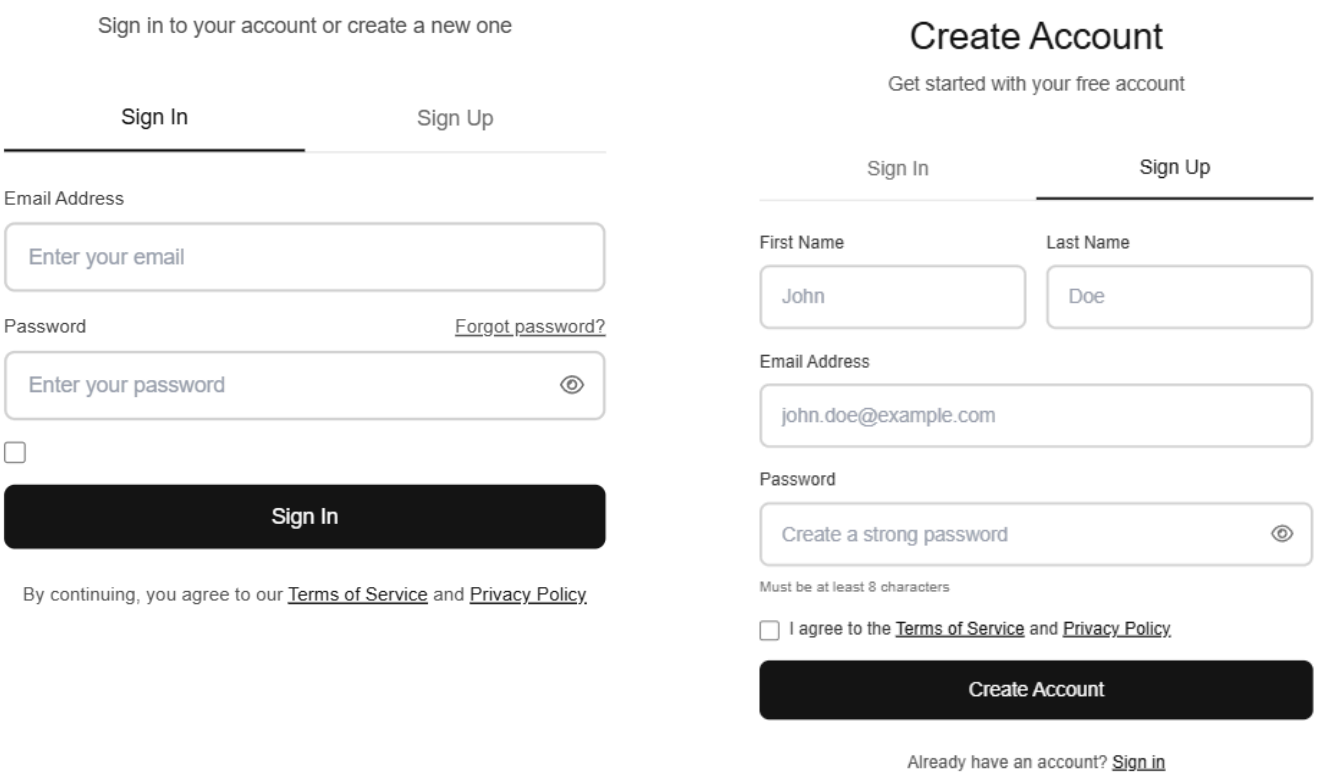


Figure 4.5.2: Authentication Page Wireframe

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation

This section describes how the Hotel Management System (HMS) was developed, including the tools used and the detailed implementation of system components. It explains the technologies adopted and how different parts of the system were coded and integrated to achieve the required functionality.

5.1.1 Tools Used

CASE Tools

Computer Aided Software Engineering (CASE) tools were used to support system design, development, and collaboration. Figma was used for designing user interfaces and wireframes. Git and GitHub were used for version control, enabling team collaboration, change tracking, and code management throughout the development process. Additional tools included VS Code as the primary code editor, pgAdmin for database management, Lucidchart for diagramming, Viber for daily communication, and Google Docs for documentation.

Programming Languages

Python was used as the primary backend programming language through the Django framework. HTML5 was used for structuring web pages, CSS3 was used for styling and responsive design (employing Flexbox and Grid layouts with a mobile-first approach), and JavaScript (ES6 standards) was used for client-side validation, interactivity, dynamic calculations, and asynchronous AJAX operations.

Database Platforms

PostgreSQL was used as the database platform for storing and managing system data. It was used both in local development (with psycopg2-binary adapter) and in the production environment on Render Cloud due to its reliability, performance, and seamless compatibility with Django ORM.

5.1.2 Implementation Details

This section explains how the Hotel Management System (HMS) was practically developed and integrated. It describes the internal structure of the system, including the implementation of modules, classes, functions, and algorithms that work together to deliver required functionalities such as room management, booking processing, user authentication, and data handling. The implementation focuses on modular design, code reusability, and maintainability, ensuring the system is efficient, scalable, and easy to enhance in the future.

Modules

The system was divided into multiple modules such as user management (authentication, signup, signin, profile), room management (display and details of room types), booking management (search, creation, confirmation, and user-specific bookings), and contact management (message submission). Additional supporting modules include cancellation policy display and administrative oversight via Django's built-in admin panel. These modules are integrated within a single Django app (hotel/) for modularity and ease of maintenance.

Classes

Classes were implemented using Django models to represent database entities such as User (extending Django's built-in authentication), Room, Booking, BookingItem, Contact Message, and Cancellation Policy. These classes define the structure of data (fields, data types, constraints) and relationships (e.g., foreign keys between Booking and Room/User) using Django ORM, ensuring clear mapping to database tables.

Procedures

Procedures were implemented within Django views to handle operations such as booking creation (processing form data, checking room availability, saving bookings), room availability checking (filtering based on dates and capacity), user authentication (login/logout handling), profile management, and contact form submission. These view functions process HTTP requests, interact with models, perform business logic, and render appropriate templates or responses.

Functions

Reusable functions were written to perform common tasks such as client-side and server-side data validation, price calculation (based on room type and stay duration), form handling (using Django forms), and dynamic updates via AJAX. Global JavaScript functions were placed in site.js, while page-specific logic was embedded or separated as needed, reducing code duplication and improving maintainability.

Methods

Methods were defined within model classes (e.g., **str** for readable representations, custom save() overrides if needed) to perform actions related to specific objects, such as saving booking details with related BookingItems, updating room status indirectly through availability queries, retrieving user-specific bookings, and generating admin panel displays.

Security

Security in the Hotel Management System (HMS) was implemented using multiple layers to protect user data, system integrity, and transactions. Django's built-in authentication framework was used to manage user login, logout, and authorization, ensuring that only authenticated users could access restricted features. User passwords were securely stored using strong hashing algorithms with salting, preventing plaintext password storage.

To protect against common web vulnerabilities, CSRF tokens were enforced on all forms to prevent cross-site request forgery attacks. Input validation and Django's template escaping mechanisms were used to reduce the risk of cross-site scripting (XSS). SQL injection attacks were prevented by using Django ORM, which automatically parameterizes database queries.

Sensitive configuration values such as secret keys and database credentials were stored in environment variables rather than hard-coded in the source code. HTTPS with SSL encryption was enabled in the production environment to secure data transmission between the client and the server. These measures collectively ensured confidentiality, integrity, and reliability of the system.

Algorithms

Basic algorithms were used for room availability checking (querying overlapping bookings and comparing dates), booking confirmation (validating non-overlapping dates, capacity, and required fields), and cost calculation (multiplying room rate by number of nights, potentially adding extras). These ensure accurate processing of hotel operations, with logic implemented in views and supported by Django's queryset methods for efficient database queries.

Authentication algorithms relied on Django's built-in authentication system, utilizing the django.contrib.auth module. User registration was handled via custom forms extending UserCreationForm or views processing signup data to create User objects with hashed passwords (using PBKDF2 by default). Login employed authenticate() to verify credentials against stored hashed passwords, followed by login() to establish session-based authentication (cookie-based sessions). Logout used the logout() function to clear sessions. Access control was enforced with decorators like @login_required on views, and permissions/groups for role-based access (e.g., admin vs. user). This provided secure, session-managed user authentication without custom hashing algorithms, leveraging Django's secure defaults for password storage and verification.

5.2 Testing

Testing was carried out to ensure that the Hotel Management System (HMS) functions correctly and meets both user and administrative requirements. The testing process involved preparing test cases based on system requirements, generating test data manually, executing the system with multiple inputs, and comparing actual results with expected outcomes.

5.2.1 Test Cases for Unit Testing

Unit testing focused on verifying individual modules and components of the system independently.

Unit Test Cases Included

- Login validation
- Account creation

- Contact form submission
- Input validation and boundary testing

Contact Form Unit Testing

Checking the contact links:

Test scenario ID	**Contact-1**	**Test case ID**	**Contact 1-A**
Description	Contact positive test	**Test priority**	High
Pre requisite	Details	**Test requisite**	NA

Execution steps:

SN	**Action**	**Inputs**	**Expected**	**Actual**	**Test result**
1	Contacts entering with blank and hit Send Message	Name: Email: Subject: Message:	Display sent failed.	Display sent failed.	Correct
2	Contacts entering with Invalid email And hit Send Message	Name: abc Email: abc Subject: subject Message: this is a message.	Display sent failed	Display sent failed	Correct

3	Contacts entering with form filed valid details and hit Send Message	Name: john Email: john@gmail.com Subject: requesting Message: I request for the booking	Display sent success	Display successful	Correct

Sign In Unit Testing

Checking the sign in links:

Test scenario ID	**Sign in-1**	**Test case ID**	**Signing 1-A**
Description	Sign in positive test	**Test priority**	High
Pre requisite	Details	**Test requisite**	NA

Checking SignUp/SignIn page:

SN	**Action**	**Inputs**	**Expected**	**Actual**	**Result**
1	Click on Sign In without entering any details	username: password:	Invalid message generated	Invalid message generated	Correct

2	Click on Sign In after entering details with out any account previously made	username: abcd134 password:*****	Invalid message generated	Invalid message generated	Correct
3	Click on Sign In after entering valid details of user account	username: john123 password: *****	Show profile and the booking details if any	Show profile and the booking details if any	Correct

Create Account Unit Testing

Checking create account page:

Test Scenario	**Create account**	**Test case id**	**Create 1-A**
Description	Create account positive test case	**Test Priority**	High
Pre Requesite	user account	**Post-Requesite**	NA

Steps:

SN	Action	Inputs	Expected	Actual	Result
1	Enter valid name and all the details for new account	Fullname: john don Username: john123 Email: john@gmail.com Password:***** Confirm Password: *****	Account created successfully. Go to SignIn page	Account created successfully. Go to Sign In page	Correct
2	Enter invalid email or other details	Fullname: john don Username: john123 Email: john.com Password:***** Confirm Password: *****	Display @ is missing in the email portion	Display @ is missing in the email portion	Correct

3	Enter the Password and the Confirm Password differently	Fullname: john don Username: john123 Email: john.com Password:**** Confirm Password: ********	Display Passwords do not match	Display Passwords do not match	Correct

5.2.2 Test Cases for System Testing

System testing verified the complete functionality of the integrated HMS modules.

Booking System Testing

Checking Book rooms:

Test Scenario	**Book rooms**	**Test Case ID**	**Book1-A**
Description	Booking positive test	**Test Priority**	High
Pre Requesite	User account	**Post-requesite**	NA

Steps:

SN	Action	Inputs	Expected	Actual	Result
1	Click on Book Now in the ROOMS page	no account details blank username: password:	Go to SignUp / Sign In page	Go to Sign Up/ Sign In page	Correct
2	Click on Book Now	Correct user details like: username/email:john123 password: ******	Go to Booking page	Go to Booking page	Correct
3	Click on Book Now	Invalid user details like: username/email: john password: ***	Show invalid username/email or password	Show invalid username/email or password	Correct

Booking After Successful Login

Booking after the account is created and sign in is successful:

Test Scenario	**Book rooms**	**Test case ID**	**Book2-A**
Description	booking positive test case	**Test Priority**	High
Pre requesite	user account	**Post-requesite**	NA

Steps:

SN	**Action**	**Inputs**	**Expected**	**Actual**	**Result**
1	Click on Book Now after the account is created and details of booking are entered	Valid Date: Check In :10/30/2025 Check Out: 10/31/2025 Room Type: Deluxe Room Available Room Numbers: 201 Any special requests:	Booking confirmed Update profile of user	Booking confirmed Update profile of user	Correct

2	Click on Book Now after the account is created and details of booking are entered blank	Valid date: Check In : Check Out: Room Type: Available Room Numbers: Any Special requests:	Display enter details	Display enter details	Correct
3	Click on Book Now after the account is created and details of booking are entered Enter the dates Invalid	Valid date: Check In :10/31/2025 Check Out: 10/21/2025 Room Type: deluxe room Available Room Numbers: 201 Any Special requests:	Cannot book	Cannot book	Correct

Sign In After Account Creation

Checking SignIn after the account is created:

Test Scenario	**Sign In**	**Test case ID**	**SignIn 1-A**
Description	Sign In positive test case	**Test Priority**	High
Pre Requesite	User account	**Post-requesite**	NA

Steps:

SN	**Action**	**Inputs**	**Expected**	**Actual**	**Result**
1	Enter valid Name and password after account is created	Username/Email : john123 Password:*****	Go to Profile page and bookings details	Go to Profile page and bookings details	Correct
2	Enter invalid details	Username: john123 Email: john.com Password:***	Display invalid username/email or password	Display invalid username/email or password	Correct

5.3 Result Analysis

The testing results indicate that the Hotel Management System performs reliably and meets both functional and user requirements.

Verification Results

Requirement	Implementation	Result
User can book a room online	Booking module (booking.html, booking.php)	Pass
Admin can manage rooms	Admin dashboard	Pass

Validation Results

- Users can successfully book rooms online.
- Admin users can manage rooms through the dashboard.
- Error handling works effectively for invalid inputs.

Validation Scenarios for Hotel Management System

Scenario	Expected Outcome	Validation Result
Guest books a room successfully	Confirmation page shown, data saved in DB	Pass

Admin updates room availability	Database reflects new status	Pass
Invalid booking dates entered	System shows proper error message	Pass
Login for wrong password	System denies access	Pass
UI elements follow design layout	Interface matches design mockups	Pass

Summary

Requirement ID	**Description**	**Validated By**	**Status**	**Comments**
R1	Room booking functionality	User/Tester	Pass	Works as expected
R2	Admin can manage rooms	Admin	Pass	Functionality verified

CHAPTER 6

CONCLUSION AND FUTURE RECOMMENDATION

6.1 Conclusion

The Hotel Management System (HMS) project was developed to automate and modernize hotel operations that are traditionally handled manually in Nepal. It centralizes core hotel functions such as reservations, check-ins, billing, and housekeeping into a single digital platform, improving efficiency and accuracy. The system was built using Django as the backend framework with PostgreSQL for the database, and HTML, CSS, and JavaScript for the frontend. Agile methodology was adopted for development, ensuring iterative progress, collaboration, and timely feature delivery. The HMS reduces human error, enhances real-time data access, and improves staff coordination, ultimately raising guest satisfaction levels. It supports scalability through cloud deployment on Render, allowing multiple property management and remote accessibility. Security measures such as password hashing (PBKDF2 with SHA-256), HTTPS, and CSRF protection ensure user data safety. Testing included both black-box and white-box approaches to validate functionality, performance, and user experience. The project successfully met its objectives of streamlining hotel processes and improving operational reliability. It also serves as a scalable foundation for small and mid-sized hotels aiming to compete in Nepal's growing tourism market. Overall, HMS demonstrates how automation and secure digital management can transform traditional hotel workflows into efficient, modern systems.

6.2 Future Recommendations

Although the hotel management system is proficient enough for real life scenarios , some things that are given below can be implemented in the future for better user experience and for a higher quality product.

- Online Payment Integration – Implement secure payment gateways (e.g., Khalti, eSewa, or Stripe) for direct online booking payments to improve convenience and reduce manual billing.
- Mobile Application Development – Build Android and iOS apps for real-time booking, notifications, and hotel management to enhance accessibility for both guests and staff.

- AI-Based Recommendation System – Introduce AI to suggest personalized room packages, discounts, or nearby services based on user preferences and booking history.
- Multi-Language and Multi-Currency Support – Add features for language localization and currency conversion to attract international tourists and expand usability.
- IoT and Smart Room Integration – Enable smart features like digital keys, automated room temperature control, and occupancy sensors for enhanced guest comfort.
- Advanced Analytics Dashboard – Develop a detailed analytics module with charts and reports to help managers make data-driven decisions on occupancy trends and revenue.
- Integration with External Platforms – Connect with Online Travel Agencies (OTAs) and review sites (e.g., Booking.com, TripAdvisor) for synchronized bookings and better exposure.

References

[1] Lucid Software Inc., *Lucidchart: Diagramming software*. [Online]. Available: https://lucid.app

[2] JGraph Ltd., *Diagrams.net: Free online diagram editor*. [Online]. Available: https://app.diagrams.net

[3] Figma Inc., *Hotel website wireframe design*. [Online]. Available: https://www.figma.com/

[4] OpenAI, *ChatGPT [Large language model]*, 2022. [Online]. Available: https://chat.openai.com

[5] DeepSeek, *DeepSeek Chat [AI model]*, 2024. [Online]. Available: https://chat.deepseek.com

[6] Grok, *Grok [Large language model]*, 2022. [Online]. Available: https://grok.ai

[7] Google, *Gemini [Large language model]*, 2024. [Online]. Available: https://gemini.google.com

[8] Marriott International, *Hotel Aloft Kathmandu Thamel*. [Online]. Available: https://www.marriott.com/en-us/hotels/ktmal-aloft-kathmandu-thamel/overview/?scid=f2ae0541-1279-4f24-b197-a979c79310b0

[9] Share Sansar, "Nepalese hotel industry: An overview," Jan. 16, 2020. [Online]. Available: https://content.sharesansar.com/photos/FEB12/NewFolder/NewFolder/january%2016%202020/Nepalese-Hotel-Industry-An-Overview-1(1).pdf

[10] Government of Nepal, *Tourism sector profile*, 2020. [Online]. Available: https://ibn.gov.np/wp-content/uploads/2020/04/Tourism-Sector-Profile.pdf

[11] ResearchGate, *Environmental analysis of tourism and hotel industries and their impact on Nepalese base structure*. [Online]. Available: https://www.researchgate.net/publication/342862787_A_Brief_Environmental_Analysis_of_Tourism_and_Hotel_Industries_and_their_Impact_on_Nepalese_Base_Structur

[12] Garima Capital, *Hotel and tourism sector*. [Online]. Available: https://garimacapital.com/assets/uploads/files/newarticleinsights/Hotel_and_Tourism_Sector.pdf

[13] Nepal Tourism Board, *Nepal Tourism Insights*, Oct. 2025 & Sept. 2025. [Online]. Available: https://trade.ntb.gov.np/downloads-cat/nepal-tourism-statistics

[14] New Business Age, "Tourist arrivals plunge in Nepal after Gen Z protests." [Online]. Available: https://www.newbusinessage.com/news/45508/tourist-arrivals-plunge-in-nepal-after-gen-z-protests/

[15] Marriott International, *Terms of Use*, Nov. 2025. [Online]. Available: https://www.marriott.com/about/terms-of-use.mi#accordion-64e1843bb8-item-8b2a2eda95

[16] Marriott International, *Privacy Statement*, Nov. 2025. [Online]. Available: https://www.marriott.com/about/privacy.mi#accordion-877ee9636a-item-68f1b358aa

[17] Hotel Landmark Nepal, *Cancellation Policy*, Nov. 2025. [Online]. Available: https://www.landmarknepal.com/cancellation-policy

[18] PrenoHQ, *How to Write an Effective Hotel Cancellation Policy*, Nov. 2025. [Online]. Available: https://prenohq.com/blog/hotel-cancellation-policy/

[19] Stratoflow, *Software Testing Process Explained: Key Stages, Types & Best Practices*, Nov. 2025. [Online]. Available: https://stratoflow.com/software-testing-process/

Appendix: UI Screenshots

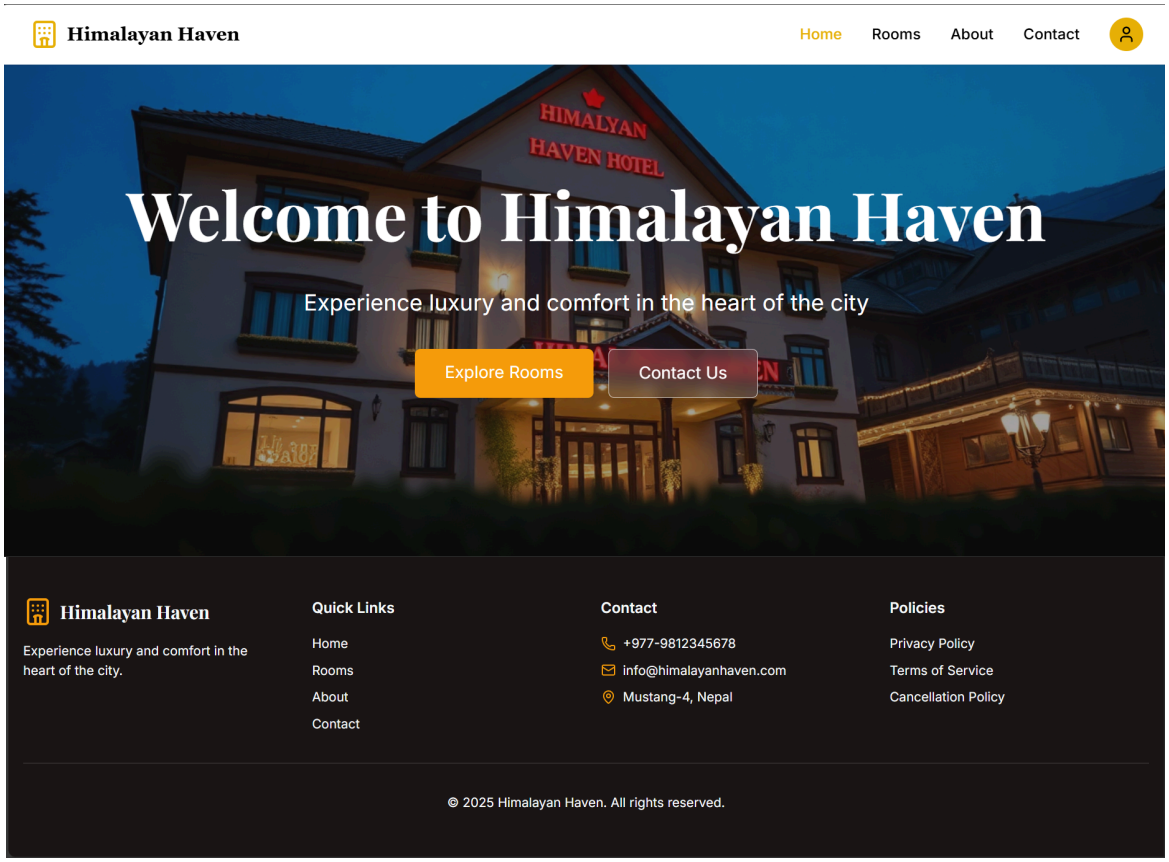


Figure 1: Home Page

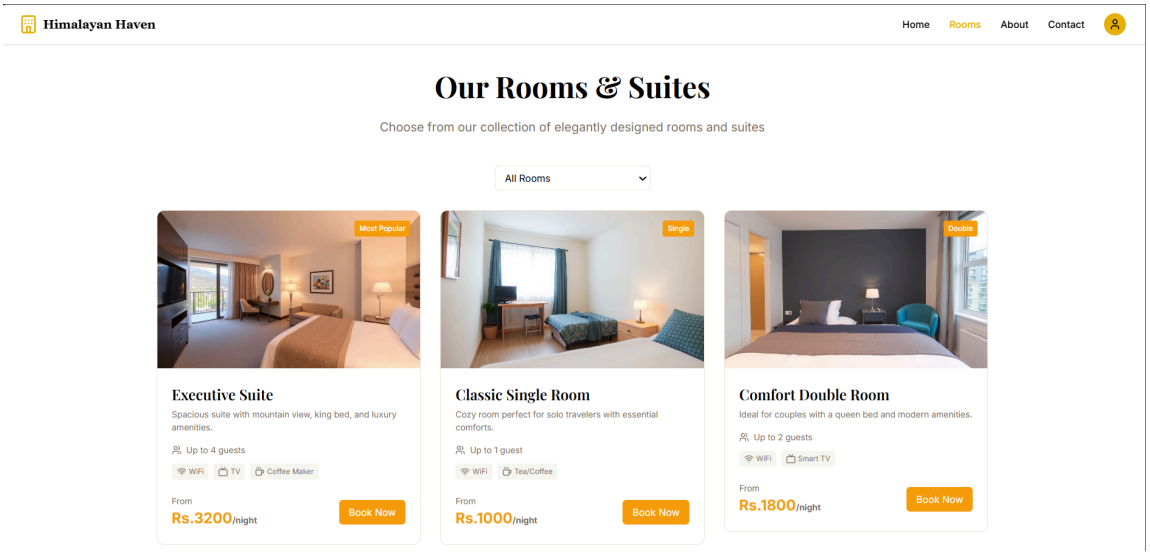


Figure 2: Room Page

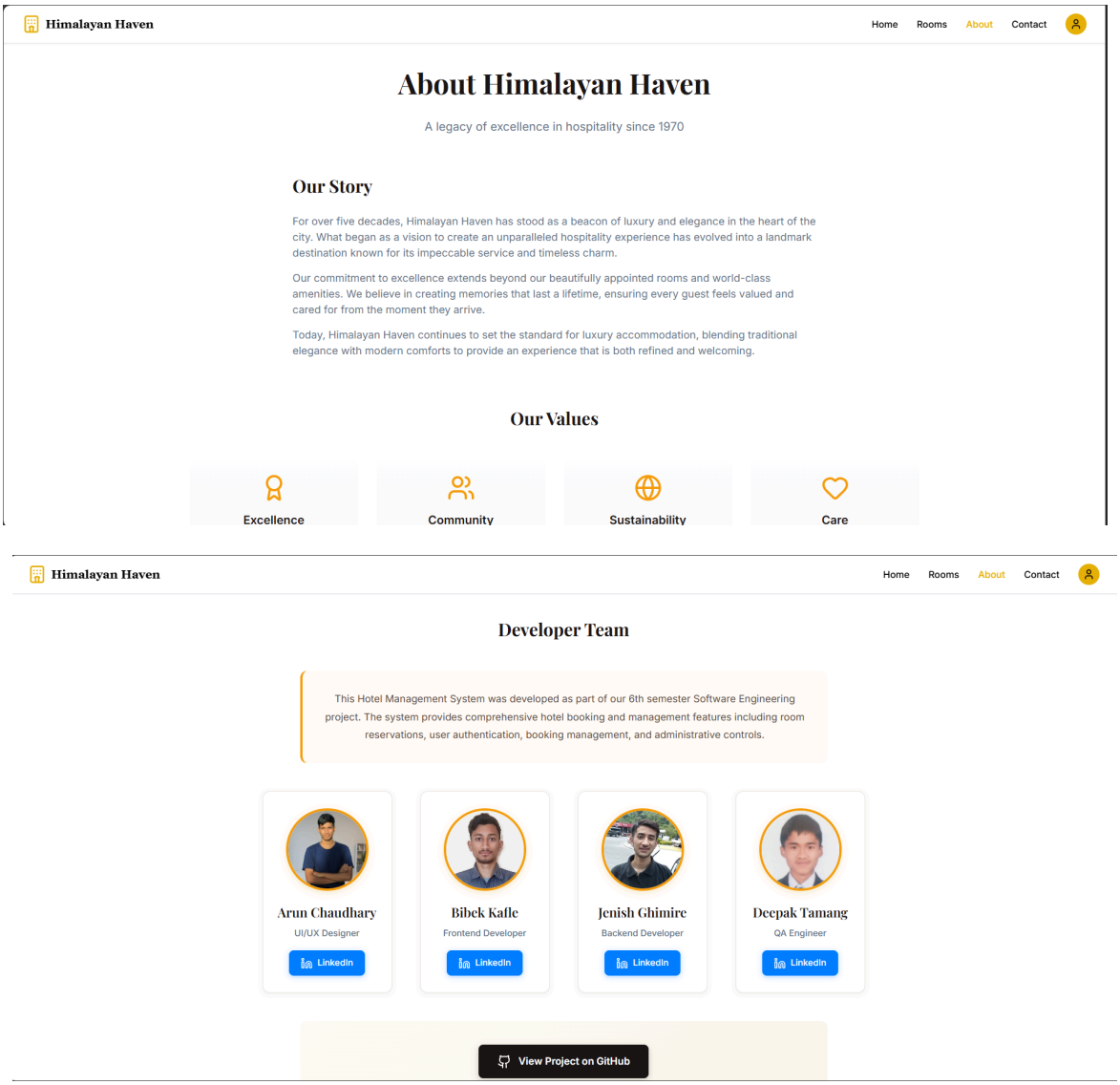


Figure 3: About Page

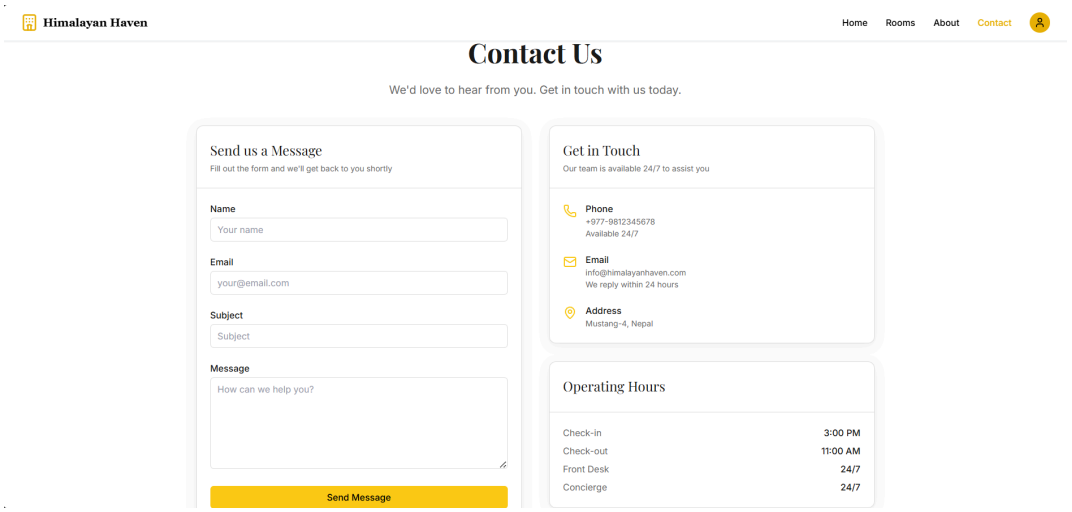
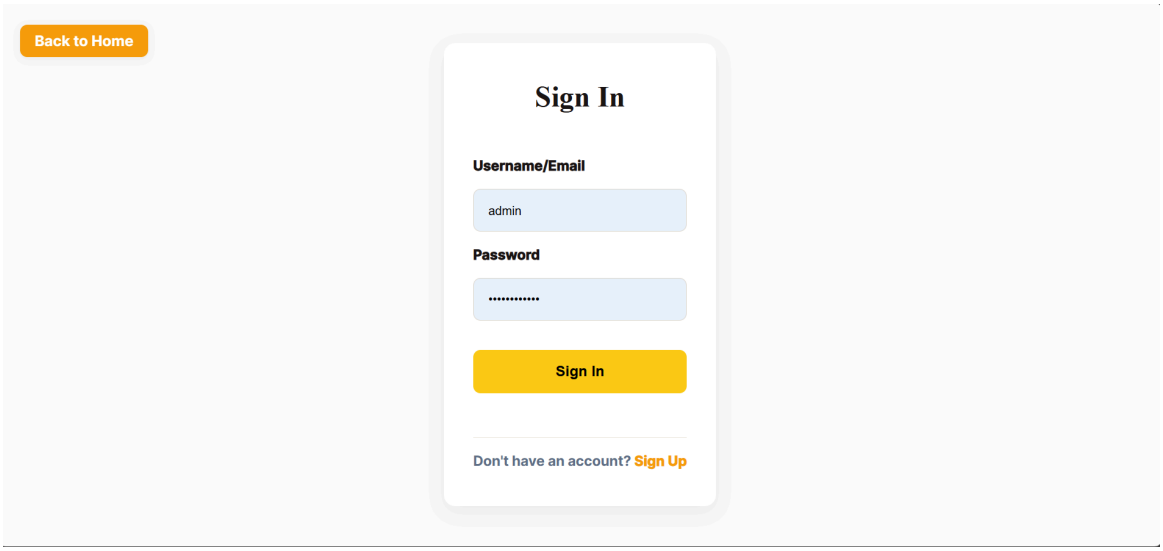


Figure 4: Contact Page



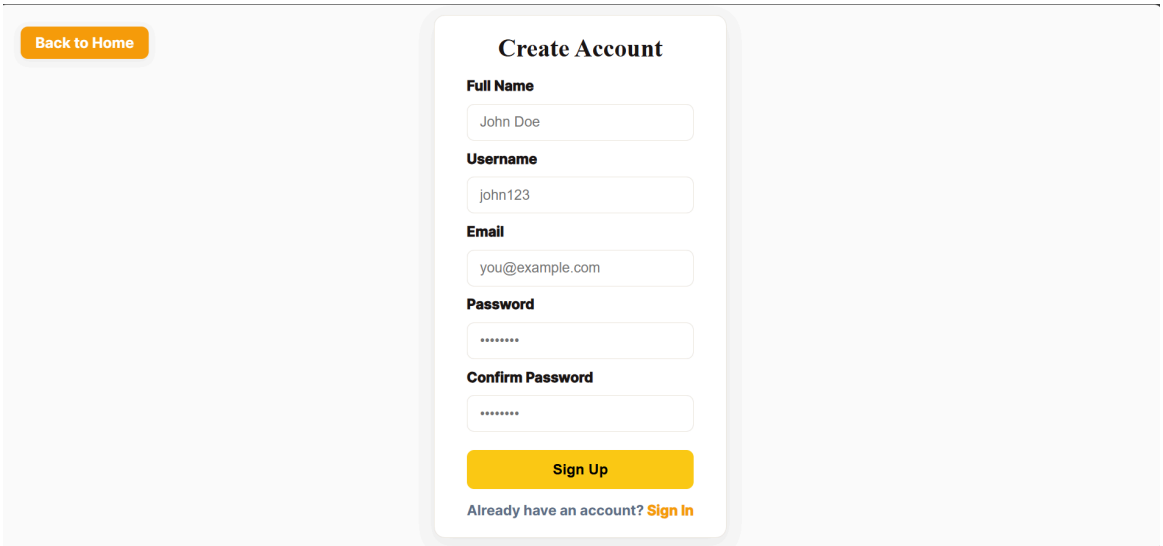


Figure 5: Authentication Page

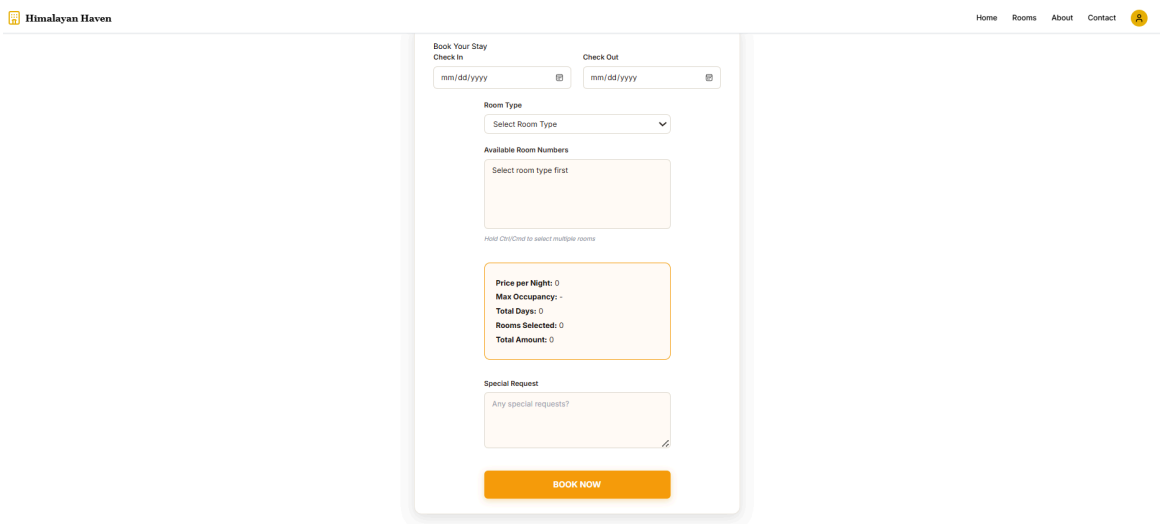


Figure 6: Booking Page

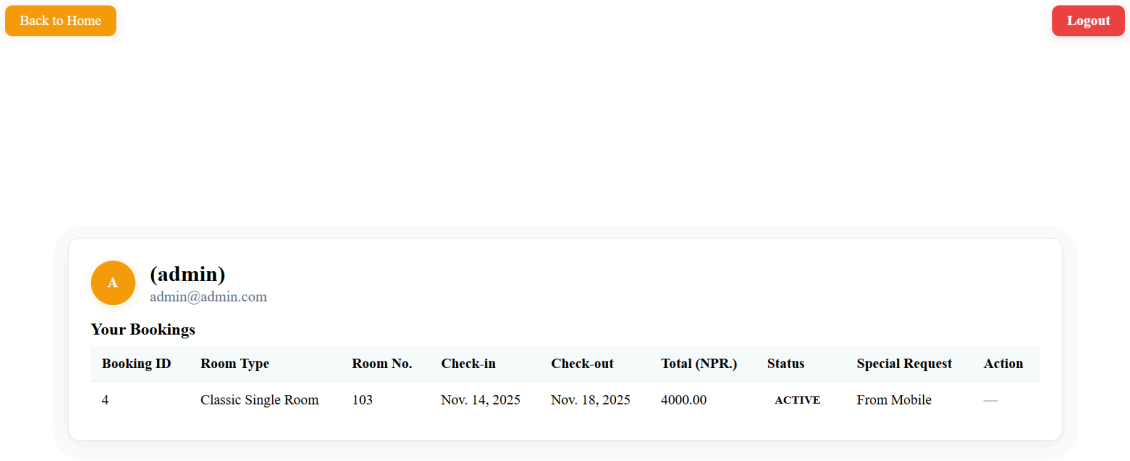


Figure 7: Booking History Page

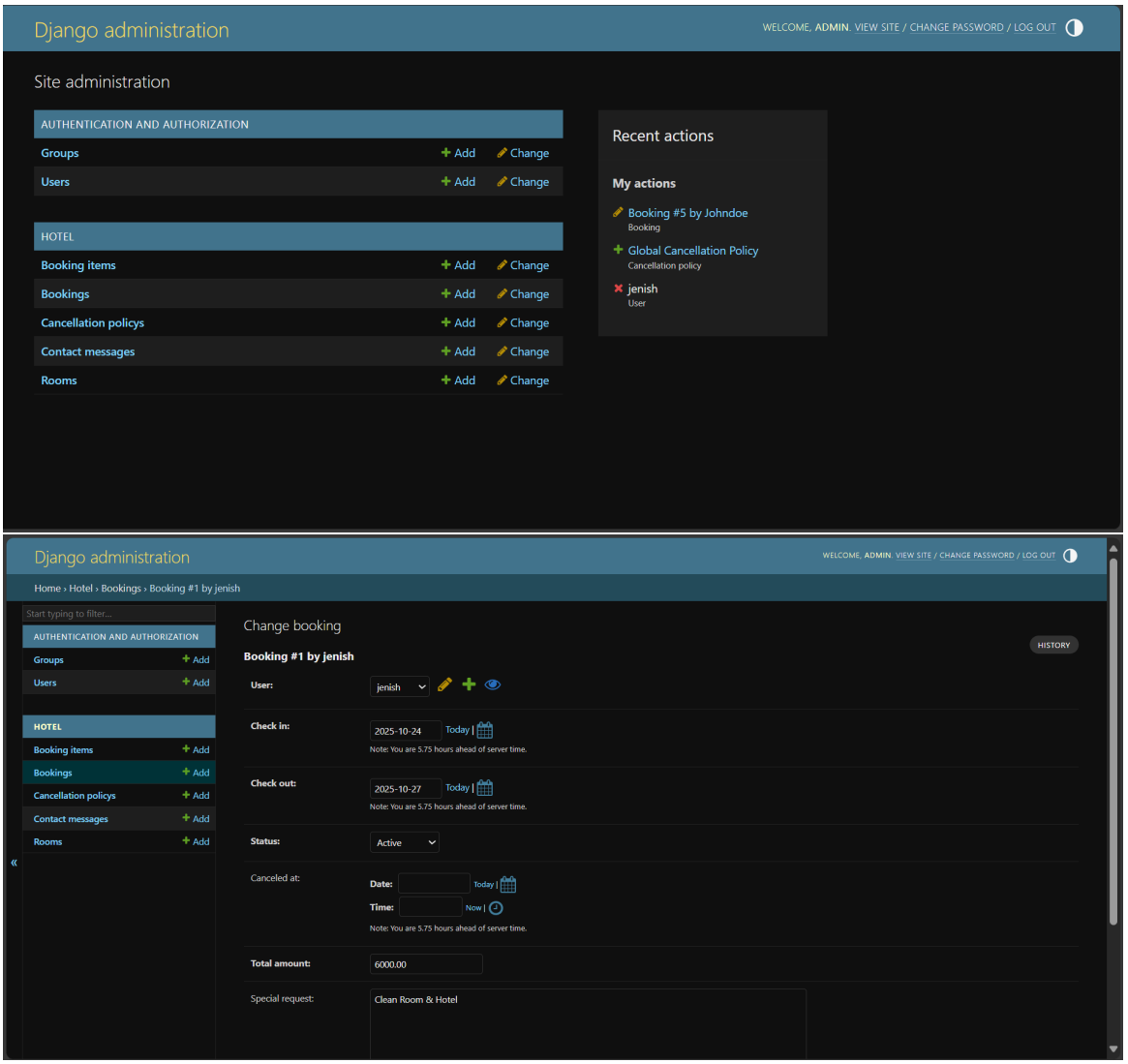


Figure 8: Admin Panel Page